IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 26 December 2024, accepted 21 January 2025, date of publication 29 January 2025, date of current version 11 March 2026.

Digital Object Identifier 10.1109/ACCESS.2025.3535922

RESEARCH ARTICLE

RPHF-GNN: Recurrent Perception of History-Future Graph Neural Networks for Temporal Knowledge Graph Reasoning

SILING FENG, ZIMIN YE, QIAN LIU, AND MENGXING HUANG, (Member, IEEE)
Hainan University, Haikou 570000, China

Corresponding author: Mengxing Huang (huangmx09@163.com)

This work was supported in part by the National Natural Science Foundation of China under Grant 62466016 and Grant 62241202, and in part by Hainan Provincial Key Research and Development Program under Grant ZDYF2022SHFZ304.

ABSTRACT TKG (Temporal Knowledge Graph) reasoning has become a hot research topic in recent years. its purpose is to predict the future by modeling historical information. However, existing research has primarily focused on comprehending the patterns and rules of historical facts, often overlooking the evolving trends in fact evolution driven by the emergence of new entities. This oversight poses challenges for models that learn entity and relation embeddings based on extensive historical information, ultimately resulting in a decrease in prediction accuracy. To address this challenge, we propose a graph-based neural network model, named RPHF-GNN (Recurrent Perception of History-future Graph Neural Networks). Specifically, RPHF-GNN divides the sequence into subgraph sequences of 'historical past' and 'historical future' at each timestep, and employs Hi-GRU (Historical-Future Information Gated Recurrent Unit) to recursively model both sequences in parallel. This allows the model to continuously perceive changes in the evolution patterns brought by unseen entities, thereby better adapting to the trends of future evolution pattern changes and enhancing the impact of Hi-GRU during the evolution process through improved Time-gate Integration Components. Additionally, in the process of constraining entity embeddings with static properties, SP-Cell (Static Perception Cell) integrates historical information from entity embeddings into the static properties to enhance the memory of the model regarding the past. It also aligns static embeddings with entity embeddings at each timestamp to optimize the static loss. We evaluate the RPHF-GNN model using six benchmark datasets, and the experimental results demonstrate significant improvement in various evaluation metrics, with the most notable enhancement reaching 1.71%.

INDEX TERMS Temporal knowledge graphs, graph representation learning, graph neural networks, knowledge embedding, event prediction.

I. INTRODUCTION

A knowledge graph is formed by a set of triples consisting of entities, relations, and objects, such as the triple (Trump, nationality, the United States). Due to its strong ability to represent facts about knowledge, it has developed rapidly in recent years. However, in the real world, knowledge is constantly evolving with the passage of time, new facts, events, and relations continuously emerge, leading to complex temporal interactions among dynamic facts [1]. Thus, temporal information is included in the knowledge graph to better simulate and understand the dynamics and complexity of the real world. This is crucial for a variety of downstream applications that are time-sensitive, including recommendation systems [2], financial analysis [3], medical fields [4], [5], and more. As shown in Figure 1, this is an example of a Temporal Knowledge Graph (TKG) composed of three adjacent timestamps, each containing a series of international political facts. The temporal dimension of the knowledge graph enables us to capture and analyze

The associate editor coordinating the review of this manuscript and approving it for publication was Juan Wang.

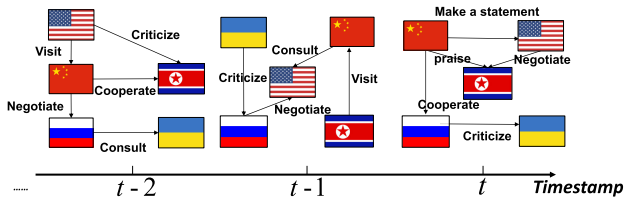

FIGURE 1. Temporal knowledge subgraphs about international political facts over time.

the temporal correlations between different countries with more accuracy. Thus, temporal knowledge graphs play a crucial role in inferring and describing the evolution of real-world facts over time in downstream applications. Currently, temporal knowledge inference methods can be broadly categorized into two types: interpolation [6], [7], [8] and extrapolation [9], [10], [11]. Interpolation aims to predict facts within a given time range ($t_0 \leq t \leq t_T$), while extrapolation focuses on forecasting future facts beyond the most recently observed timestamp $\left(t > t_T\right)$. Clearly, the latter presents a more challenging task and has important implications for factual predictions such as the prediction of natural disasters [12] and risk prevention [13]. In this paper, our research primarily concentrates on extrapolation-based reasoning.

For accurate predictions, it is essential to delve into historical facts understand the trend of the evolution of facts. In the real world, a wealth of newly emerging facts can influence the direction of entity and relation evolution. Facts also follow different evolutionary patterns at different times [14]. For instance, the facts announced during the Obama presidency (United States, re-establishment of diplomatic relations, Cuba, 2014-12-17) marked the easing of long-standing tensions between the two countries. However, with the coming to power of President Trump, a series of deteriorating relations between the two countries have occurred, such as (United States, close embassy, Cuba, 2017-09-29) and (United States, restrict travel, Cuba, 2020-01-10), and this positive trend has suffered a reversal. The above example illustrates that the emergence of new facts can lead to a shift in the relationship between the two countries from positive to negative. Therefore, an intuitive idea is that continuously perceiving changes brought by unseen entities can help the model better adapt to the evolving patterns of future trends.

Recently, various methods have made heuristic attempts to extract pertinent historical information for different queries. However, none of them have considered the assistance of evolutionary patterns to the models. RE-GCN [9] introduces the capture of temporal graph dependencies to represent the evolution of entities and relations. RE-NET [10] encodes historical facts related to a given entity. RE-GAT [15] uses the Event Focus Module to model the relationships between concurrent events. Tlogic [16] automatically extracts temporal logical rules from temporal knowledge graphs based on temporal random walks. DHE-TKG [17] captures higher-order correlations between entities by combining low-order and high-order information, and dynamically integrates temporal location information through dynamic meta-embedding techniques to determine the importance of input historical graphs. However, the above models only focuses on the modeling of historical information and ignores the evolution pattern implied in the sequence. Additionally, although RE-H^{2}AN [18] employs a hierarchical hypergraph recurrent attention mechanism and evolutionary learning for deducing intricate relational patterns in temporal knowledge graphs, it overlooks the assistance provided by static entity properties during the evolution of entities. Although RE-GCN [9] imposes constraints on entities using static properties, it overlooks the dependencies of static properties on entity embeddings.

To address these challenges, a model referred to as RPHF-GNN was proposed. Specifically, the model employs Hi-GRU (Historical-Future Information Gated Recurrent Unit) to recursively model the sequential dependencies of historical information while continuously extracting entity information involved in the 'historical future' for parallel modeling, in order to adapt to the changing trends of future evolutionary patterns. Additionally, for the sequential patterns between adjacent subgraph combinations, we augment the influence of Hi-GRU during the evolution process and dependencies between sequences by improving the Time-gate Integration Components. Finally, in the process of merging static properties of entities, this model utilizes the SP-Cell (Static Perception Cell) component to absorb knowledge from external evolving entities, integrating it into static properties. This enhances the memory of the model regarding historical information and alleviates potential issues related to the loss of information. Moreover, the component adjusts the static embeddings at each timestamp to align more effectively with entity embeddings, optimizing static loss.

In summary, this paper makes the following contributions:

- We propose an evolutionary representation learning model, RPHF-GNN, for Temporal Knowledge Graph reasoning. In this model, the two sequences of 'historical past' and 'historical future' in the subgraph combination are modeled recursively in parallel to adapt to different evolutionary patterns, and the static properties are adjusted to adapt to various evolutionary sequences within the TKG. To the best of our knowledge, this is the first instance of integrating these features into TKG reasoning.
- The model employs an evolutionary module to recursively model the sequential dependencies within subgraph combinations, perceiving unseen entities and adapting to the trends of future evolutionary pattern changes. Additionally, during the evolution process, historical information is integrated to evolve static properties, enhancing the memory of the model with respect to historical information and optimizing static loss.

- Through an extensive set of experiments, we have demonstrated that RPHF-GNN outperforms both entity prediction and relation prediction on six public TKG datasets.

The remainder of this paper is structured as follows: Section II demonstrates an overview of related work, Section III provides a detailed description of the proposed model, Section IV presents the analysis of experimental results, Section V. summarizes the conclusions, and Section VI presents the acknowledgments for this research.

II. RELATED WORKS

In previous static KG reasoning methods, modeling typically focused on handling triplets, often omitting considerations of time and dimensionality. TransE [19] models relational data by translating vectors in vector space. DistMult [20] learns embeddings for entities and relations using matrix factorization. ComplEx [21] proposes a link prediction model based on complex-valued vectors, utilizing complex multiplication and dot product operations to calculate similarity between entities and relations for link prediction. Methods based on convolutional networks, such as ConvE [22] and ConvTransE [23], capture features of entities and relations through convolutional operations. They introduce convolutional kernels for these operations, which facilitates representation learning in the embedding space. However, these methods fail to capture the temporal evolution of information, thus having limitations in effectively predicting future facts. Therefore, researchers have endeavored to perform reasoning on missing or future historical facts by incorporating time information into Knowledge Graphs, which is known as Temporal Knowledge Graph reasoning [24].

Regarding TKG reasoning, it can be categorized into two settings: interpolation and extrapolation. In the interpolation setting, TTransE [25] extends the traditional TransE to the temporal scenario, associating each relation with a specific time period or timestamp and treating relations and time as translations between entities. Similarly, HyTE [8] projects relation and time vectors onto corresponding hyperplanes to obtain representations of relations at different times. However, these methods do not consider more time dependencies and historical data, thus they cannot predict future facts. Therefore, the extrapolation setting of temporal knowledge graph has been gradually developed in recent years. RE-NET [10] employs a self-recurrent approach based on Recurrent Neural Networks (RNNs) [26] to perform reasoning on temporal knowledge graphs in time sequences. RE-GCN [9] recursively models temporal subgraph sequences and uses entity static properties to constrain entity representations. RE-GAT [15] jointly encodes using RNNs and GNNs, employing an Event Attention Module to model associations among concurrent events. Tlogic [16] constructs temporal logic-based query paths. RE-H^{2}AN [18] employs a Hierarchical Hypergraph Recurrent Attention Network and Hierarchical Evolutionary Learning to infer complex event relationships within Temporal Knowledge Graphs. FITCARL [27] integrates few-shot learning with reinforcement learning, utilizing a policy network for navigating TKGs to make predictions. PPT [28] utilizes pre-trained language models to obtain semantic information for completion tasks. DHE-TKG [17] captures higher-order correlations between entities by combining low-order and high-order information, and dynamically integrates temporal location information through dynamic meta-embedding techniques to determine the importance of input historical graphs. However, these methods fail to consider the pattern changes brought about by the continuous emergence of unseen entities, which makes it difficult for them to sustainably adapt to new evolutionary patterns. Moreover, they lack exploration of the dependency relationship between entity embeddings and static properties. In comparison, the RPHF-GNN model recursively models the two sequences in the subgraph combination sequence in parallel, perceives the evolutionary pattern changes brought by unseen entities, and adjusts static properties by integrating entity information throughout the entire evolution process. These measures significantly enhance the performance of model.

III. METHODOLOGY

In this section, we introduce the proposed RPHF-GNN method. We start by presenting the symbols and definitions used. Then, we provide an overview of the overall framework of the model and an introduction to the three modules of the model.

A. DEFINITIONS

In this paper, A TKG can be regarded as a sequence of subgraphs sorted by timestamp, i.e.,$G = \{G_1, G_2, \ldots, G_t, \ldots\}$. Each subgraphs in G can be represented as $G_t = (V, R, T_t)$, where V and R respectively represent sets of entities and relations, and T_t is the set of facts at timestamp t. The fact in T_t can be represented as a quadruple, (s, r, o, t) where $s, o \in V$ and $r \in R$. This denotes a fact of relation, with r serving as the subject entity and s as the object entity o at timestamp t. For each quadruple (s, r, o, t), an inverse relation quadruple (o, r^{-1}, s, t) is often added to the dataset. TKG reasoning involves two main tasks: entity prediction $(s, r, ?, t)$ and relation prediction $(s, ?, o, t)$. These tasks aim to predict missing entities or relations at timestamp t. For each prediction at timestamp $t + 1$, the prediction of the facts depends on the KGs from the latest n timestamps $(\{G_{t-n+1}, \ldots G_t\})$. The information of historical KGs is embedded in the matrices $H_t \in \mathbb{R}^{|V| \times d_V}$ and $R_t \in \mathbb{R}^{|R| \times d_R}$, respectively representing the subjects, objects, and relations at timestamp t (where d is the dimension of the embedding vectors).

B. MODEL OVERVIEW

As shown in Figure 2, the overall framework of RPHF-GNN consists of three modules. The Input Processing Module divides timestamps into subgraph combinations and constructs a static graph. The Evolutionary Learning Module

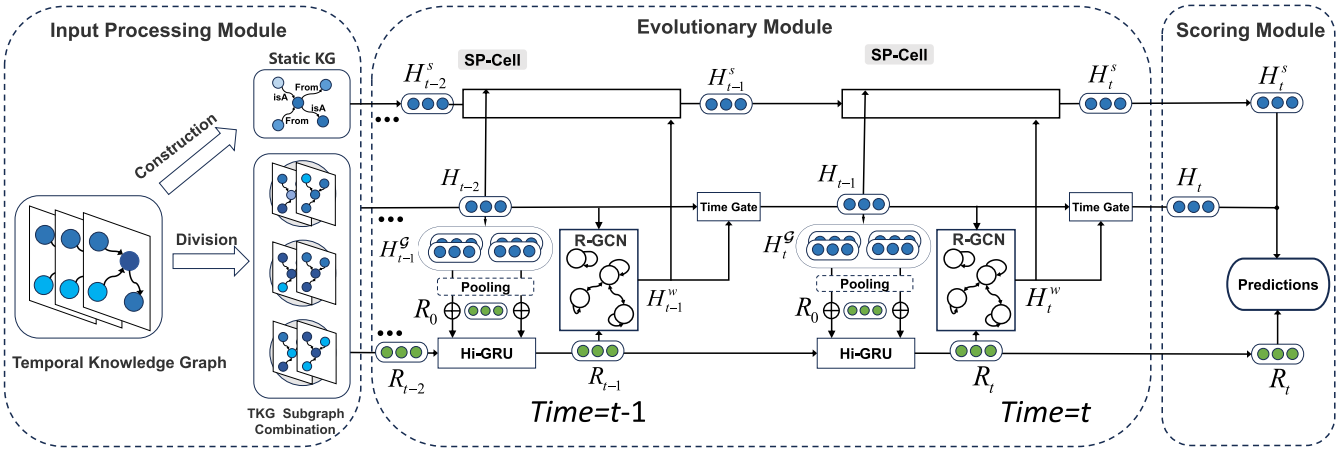

FIGURE 2. Illustration of the proposed RPHF-GNN model for temporal reasoning at timestamp $t+1$.

recursively models the sequential dependencies of entities and relationships, while continuously extracting entity information involved in the 'historical future' for parallel modeling, to adapt to the changing trends of future evolutionary patterns. Furthermore, this module takes into account the evolution of static properties to enhance the learning of entities with static background knowledge. Finally, the Scoring Module performs extrapolation reasoning by scoring the embeddings of entities and relations evolved from the last timestamp, thereby completing the prediction task.

C. INPUT PROCESSING MODULE

In TKGs, alterations in timestamps indicate modifications in information within subgraphs. Traditional methods often handle the sequential relations between timestamps and their structural dependencies by only considering historical information. To continuously perceive unseen entities and obtain information about the trends of evolutionary pattern changes, we combine each subgraph in the TKG with its adjacent subsequent subgraph. Formally, the subgraph combination can be represented as follows:

$$\mathcal{G}_t = \{G_t, G_{t+1}\}, \tag{1}$$

where G_t is the subgraph with the timestamp sequentially adjacent. Furthermore, we construct a SKG (Static Knowledge Graph) using TKG, which contains comprehensive information spanning multiple timestamps. They serve as background knowledge for the TKG, helping the model to obtain more accurate evolutionary representations. Note that at timestamp t, since no future information is available, we loop through the subgraph sequence, using G_1 as a substitute.

In simple terms, the Input Processing Module tracks changes in information by combining subgraphs from adjacent time points, and the static knowledge graph helps the model predict future trends more accurately by integrating data from multiple time points.

D. EVOLUTIONARY MODULE

The purpose of this module is to achieve more accurate representations of entities and relationships by learning from the subgraph combinations output by the Input Processing Module, as well as by incorporating constraints imposed by the evolution of static properties in the static graph. This module consists of R-GCN [29](Relation-GCN), Hi-GRU, a Time-gate Integration Component, and a static property evolution component. The R-GCN learns the structural relations between concurrent facts in KGs at each timestamp and performs aggregation. The Hi-GRU module recursively models two sequences within the subgraph combination sequence in parallel, perceiving the evolutionary pattern changes brought about by unseen entities, and integrates a temporal gating mechanism to obtain the evolutionary embeddings of related entities at each timestamp. The static property evolution component perceives and constrains the entity evolution embeddings for each timestamp.

In simple terms, the Evolutionary Module conducts an in-depth analysis of the structure and historical data of TKGs, enabling the model to comprehend the changing trends in evolutionary patterns and to evolve entity and relationship embeddings.

1) AGGREGATE EMBEDDING OF CONCURRENT FACTS

As each knowledge graph is a multi-relational graph, and Graph Neural Networks (GNNs) have strong expressive capabilities for structured data, we use a ω layer R-GCN that is able to perceive different relations to model concurrent facts among entities within the same timestamp and aggregate the influence from neighboring facts. Additionally, for each fact, we append an inverse relation quadruple. Specifically, we use the composition operation of concatenating the vector representations of the object and the subject to capture the translational property between the subject and the object. Under TKGs with timestamp t, the object o

obtains its embeddings at layer $l \in [0, \omega - 1]$ from the corresponding subjects and relations in the quadruples under a message-passing framework at layer l. It then learns its vector representations at the $l+1$ layer, following as:

$$h_{o,t}^{(l+1)} = RReLu\left(\frac{1}{C_o}\sum_{(s,r,o)\in G_t} W_1(h_{s,t}^{(l)} + r_t) + W_2 h_{o,t}^{(l)}\right), \tag{2}$$

where $h_{s,t}^{(l)}$, $h_{o,t}^{(l)}$, r_t denote the l^{th} layer vector embeddings of the subject s, object o, and relation r at timestamp t, respectively. W_1 and W_2 are the learnable parameters for aggregating features in the l^{th} layer. For entities that do not exist in G_t at timestamp t, only self-loop operations are performed. Additionally, C_o is a normalizing factor equal to the indegree of the object o, and $RReLu(\cdot)$ represents the RReLu activation function used for the aggregation operation.

2) HISTORICAL FACTS EMBEDDING

For the subject s and o in the facts, the sequential relation of historical facts implies their evolution patterns. In order to cover as many historical facts as possible,the model needs to consider the adjacent sequences of facts over time. In R-GCNs, using the entity embedding matrix from the previous timestamp as input for the current timestamp can establish a sequence of facts, but this may lead to over-smoothing because it stacks all layers of the R-GCN together, hindering the update of weights [30], Thus, in this module, we use a Time-gate Integration Component [9] to address these issues and obtain the entity embedding matrix H_t at timestamp t. Formally:

$$H_t = U_t \otimes H_t^w + (1 - U_t) \otimes H_{t-1}, \tag{3}$$

where $\otimes$ denotes the Hadamard product operation. In the previous model [9], the Time-gate Integration Components only considered the features of the evolving entity embedding matrix H_{t-1} within the sequence, while overlooking the equally important features within the entity embedding matrix H_t^w output by R-GCN. Moreover, refining the weighting of features within the Time-gate Integration Component allows for a more comprehensive integration of entity characteristics, thereby influencing the ability of Hi-GRU to model the sequential dependencies between subgraphs with greater precision and accuracy. Therefore, the Time-gate Integration Components $U_t \in \mathbb{R}^{|V|\times d_V}$ perform a nonlinear transformation as follows:

$$U_t = \sigma(W_3 H_{t-1} + W_4 H_t^w + \beta_U), \tag{4}$$

where $\sigma(\cdot)$ represents the sigmoid function, $W_3, W_4 \in \mathbb{R}^{d\times d}$ are the learnable parameter for weight matrix, and β_U is the biased term.

3) SEQUENTIAL PERCEPTION OF HISTORY-FUTURE FACTS

To address the challenge of evolving pattern changes brought about by the continuous emergence of unseen entities in time series data, an intuitive approach is to continually perceive the changes introduced by these invisible entities, which can assist the model in better adapting to the ever-changing patterns. Therefore, in addition to recursively modeling the sequential dependencies of historical information, it is also necessary to continuously extract entity information involved in the 'historical future' for parallel modeling, ensuring that the model can adapt to the changing trends in evolutionary patterns within the sequence. Consequently, we designed the Hi-GRU (Historical-Future Information Gated Recurrent Unit) component to recursively model both sequences in parallel, thereby tackling this challenge. This design leverages the strengths of GRU [31](Gate Recurrent Unit) in processing sequential data, iteratively updating the representation of the relationship embedding matrix within subgraph combinations, which is crucial for capturing the dynamics of evolutionary patterns in time series data. When GRU solely models the sequential dependencies within the historical sequence, it is influenced by the relation matrices originating from the outputs of forward neighboring subgraphs and the entity information involved in the iterative process of relation updates. To obtain the input for the GRU, the module performs mean pooling on the evolving embedding matrix of r-related entities, as defined by $V_{r,t} = \{i \mid (i, r, o, t) or (s, r, i, t) \in G_t\}$. Formally:

$$\vec{r}_t^{\,\prime} = [pooling(H_{t-1,V_{r,t}}); \vec{r}], \tag{5}$$

where $\vec{r}$ denotes the relation embeddings in the input matrices of the first historical subgraph, $H_{t-1,V_{r,t}}$ is a set that saves the related entities of specific relation at the timestamp $t-1$, and $[;]$ represents the vector concatenation operation. Then, the update gate z_t, reset gate r_t, and hidden state gate R_t in GRU are calculated:

$$z_t = \sigma(W_z[R_{t-1}, R_t^{\prime}]), \tag{6}$$

$$r_t = \sigma(W_r[R_{t-1}, R_t^{\prime}]), \tag{7}$$

$$\tilde{R}_t = tanh(W_R[r_t \odot R_{t-1}, R_t^{\prime}]), \tag{8}$$

$$R_t = (1 - z_t) \odot R_{t-1} + z_t \odot \tilde{R}_t, \tag{9}$$

where $R_t^{\prime} \in \mathbb{R}^{|R|\times 2d_R}$ is composed of $\vec{r}_t^{\,\prime}$ of all the relations and operator $\odot$ denotes the dot product. Additionally, $\sigma(\cdot)$ represents the sigmoid activation function, $tanh(\cdot)$ is the hyperbolic tangent activation function, and $W_z, W_r, W_R \in \mathbb{R}^{d\times d}$ are all the learnable parameter for weight matrix.

In modeling the sequential dependencies of facts in the 'historical future', we apply the same GRU network to the subgraphs to perceive their features. Specifically, similar to Eq.(5), we simultaneously perform mean pooling on the embeddings of r-related entities $H_{t,V_{r,t}}^{\mathcal{G}}$ for both subgraphs in the subgraph combination $\mathcal{G}_t$ at timestamp t to obtain the Hi-GRU input:

$$\vec{r}_t^{\,\prime} = [pooling(H_{t,V_{r,t}}^{\mathcal{G}}); \vec{r}], \tag{10}$$

$$\vec{r}_{t+1}^{\,\prime} = [pooling(H_{t,V_{r,t+1}}^{\mathcal{G}}); \vec{r}], \tag{11}$$

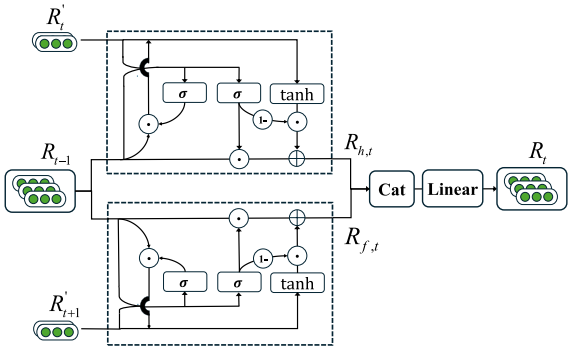

FIGURE 3. Illustration of the historical-future information gated recurrent unit.

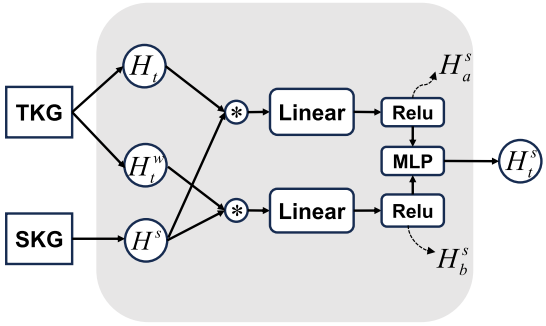

FIGURE 4. Illustration of the static perception cell.

where $\vec{r}_t'$ and $\vec{r}_{t+1}'$ are the GRU inputs, used for modeling the two sequences respectively. Thus, we expand Eqs.(8) and (9) into Eqs.(12), (13), and Eqs.(14), (15), as follows:

$$\tilde{R}_{h,t} = tanh(W_H[r_{h,t} \odot R_{t-1}, R_t']), \tag{12}$$

$$\tilde{R}_{f,t} = tanh(W_F[r_{f,t} \odot R_{t-1}, R_{t+1}']), \tag{13}$$

$$R_{h,t} = (1 - z_{h,t}) \odot R_{t-1} + z_{h,t} \odot \tilde{R}_{h,t}, \tag{14}$$

$$R_{f,t} = (1 - z_{f,t}) \odot R_{t-1} + z_{f,t} \odot \tilde{R}_{f,t}, \tag{15}$$

where $R_{h,t}$ controls the sequence-related modeling of historical information, $R_{f,t}$ continuously obtains unseen entities from the 'historical future' to facilitate the adaptation of model to the changing trends of evolutionary patterns. Moreover, $r_{h,t}$ and $z_{h,t}$ follow the same form as Eqs.(6) and (7), with the input being identical to $\tilde{R}_{h,t}$. Both W_H and W_F are weight matrices. To integrate the acquired features, we transform two state vectors into an output vector R_t by concatenating them through a linear layer, follow as:

$$R_t = W_5[R_{h,t}; R_{f,t}] + \beta_t, \tag{16}$$

where W_5 is the parameter matrix and β_t is the biased term.

4) STATIC EVOLUTIONARY PROPERTIES

In the evolution module of the model, the static properties of entities are treated as background knowledge within the TKG. We construct the SKG for the ICEWS series of datasets by extracting static property information from entity strings. The SKG contains comprehensive background information spanning multiple timestamps, introducing 'isA' and 'From' relations which serve as fundamental background knowledge for the TKG. To model the static graph, we first employ a one-layer R-GCN to encode the facts present in the SKG:

$$\vec{h}_n^{(s)} = ReLU\left(\frac{1}{c_n} \sum_{(r^{(s)},m),\exists(n,m,r^{(s)})\in G^{(s)}} W_6 \vec{h}_n^{(rs)}(m)\right), \tag{17}$$

where $\vec{h}_n^{(s)}$ is the n^{th} lines of H^s, which is the output, and $\vec{h}_m^{(rs)}$ is the m^{th} line of H'^s, which is randomly initialized input embedding matrices. Furthermore, c_n is a normalizing factor, W_6 is the matrix of relation parameters, and $\sigma(\cdot)$ is the ReLU function. However, if the model does not update the static properties that constrain entity embeddings in accordance with the evolution of entities, it cannot fully utilize historical information. Therefore, we have introduced the SP-Cell to model the dependencies between entity embeddings and static properties, integrating these embeddings into static properties during the fusion process. This enhances the memory of historical information and mitigates potential information loss. Additionally, the model adjusts the static embeddings at each timestamp to align more effectively with entity embeddings, thereby optimizing the static loss.

The structure of SP-Cell is illustrated in Figure 4. Its input consists of entity embeddings H_t and H_t^w for the evolution of TKGs, as well as the static properties H^s in SKG. The output yields the statically evolved properties H_t^s, incorporating external dynamic features at timestamp t. Specifically, at each timestamp, SP-Cell couples the entity evolution embeddings H_t and H_t^w with the static property H^s through linear layers, so that the static property perceives the evolutionary features separately. Finally, after applying the activation function, we use a double-layer MLP (multi-layer perceptron) to fuse the feature association variables H_a^s and H_b^s between entity embeddings and static properties.

Formally:

$$H_a^s = ReLU((H^s, H_t)W_a^s + \beta_a), \tag{18}$$

$$H_b^s = ReLU((H^s, H_t^w)W_b^s + \beta_b), \tag{19}$$

$$H_t^s = MLP[H_a^s; H_b^s], \tag{20}$$

where W_a^s, W_b^s are linear transformations, β_a, β_b are the biased terms, [;] is the concatenation, and $ReLU(\cdot)$ is the ReLU activation function. To capture the learning of static properties by reflecting on the entity embedding matrix, we impose a constraint that the angle between the evolution embedding and static property embedding for entities with the same timestamp does not exceed a certain threshold. As entity embeddings accumulate more facts over the course of evolution, the threshold increases over time. Thus, it is defined as:

$$\theta_x = \min(\psi x, 90°), \tag{21}$$

where ψ represents the ascending pace of the angle, and $x \in [0, 1, \ldots, k]$. The maximum threshold of the angle is

set to 90°. The loss of the static graph constraint component at timestamp t is defined as:

$$L_x^s = \sum_{i=0}^{|V|-1} \max\{cos\theta_x - cos(H_{t-k+i}^s, H_{t-k+i}), 0\}, \quad (22)$$

where H_{t-k+i}^s and H_{t-k+i} respectively represent the entity embedding matrix and the static embedding matrix on the same timestamp, and $cos(H_{t-k+i}^s, H_{t-k+i})$ represents the cosine value of the angle, noted that this value should be greater than $cos(\theta_x)$. The loss of the static graph constraint component is $L^s = \sum_{x=0}^{k} L_x^s$.

E. SCORING FUNCTION AND LEARNING OBJECTIVE

After traversing all evolution modules, we decode the output entity and relation embedding matrices to obtain probability scores for candidate triples. Previous studies [22], [32], [33] have demonstrated that Graph Convolutional Networks (GCNs) with convolutional scoring functions perform well in knowledge graph reasoning. Taking into account the translational properties of entity and relation representations in Eq. (2), we choose ConvTransE [23] as the decoder for the Scoring Module of the model. It effectively perceives relations between entities and their neighbor relations in the knowledge graph, thereby enhancing the decoding and reasoning process. Then, the probability vector of the fact object is as follows:

$$\vec{p}(o \mid H_t, R_t, s, r) = \sigma(H_t \text{ConvTransE}(s_t, r_t)), \quad (23)$$

$$\vec{p}(r \mid H_t, R_t, s, o) = \sigma(R_t \text{ConvTransE}(s_t, o_t)), \quad (24)$$

where s_t and o_t are embedding vectors of entity embedding matrix H_t, r_t are embedding vectors of relational embedding matrix R_t, and $\sigma(\cdot)$ is the sigmoid function. Both entity prediction and relation prediction can be viewed as multi-label learning problems, where each label represents a specific fact object entity or relation. We use $y_{t+1}^e \in \mathbb{R}^{|V|}$ and $y_{t+1}^r \in \mathbb{R}^{|R|}$ to denote the vector of labels for fact entity prediction and relation prediction task at the timestamp $t+1$. Formally:

$$L^e = \sum_{t=0}^{T-1} \sum_{(s,r,o,t+1)\in G_{t+1}} \sum_{i=0}^{|V|-1} y_{t+1,i}^e \log p_i(o|H_t, R_t, s, r), \quad (25)$$

$$L^r = \sum_{t=0}^{T-1} \sum_{(s,r,o,t+1)\in G_{t+1}} \sum_{i=0}^{|R|-1} y_{t+1,i}^r \log p_i(r|H_t, R_t, s, o), \quad (26)$$

where T denotes the number of timestamps in the training dataset. Note that the elements of vectors y_{t+1}^e and y_{t+1}^r are 1 for facts that do occur, and 0 otherwise. Thus, the final loss denotes as:

$$L = \alpha L^e + (1-\alpha) L^r + L^s, \quad (27)$$

where α is the task-weight that handle the loss terms.

In simple terms, the scoring module decodes entity and relationship embeddings, uses ConvTransE to score potential triples, and treats predictions as multi-label learning tasks. The total loss includes entity loss, relationship loss, and static graph loss, balanced by a parameter α.

IV. EXPERIMENTS

In this section, we evaluate the performance of RPHF-GNN model with six public available fact datasets. We begin by providing a detailed explanation of our experimental settings, which include the datasets, evaluation metrics, and baseline methods. Then, we demonstrate the performance of the model and analyze the experimental results.

A. EXPERIMENTAL SETUP

1) DATASETS

We use six fact-based TKGs datasets which record facts with timestamps, namely, ICEWS18 [10], ICEWS14 [6], ICEWS05-15 [6], WIKI [34], YAGO [35] and GDELT [36]. We strictly adhere to the division of the dataset into training, validation, and test sets, partitioning the data based on timestamps into training (80%), validation (10%), and test sets (10%). Furthermore, we use only the test set for the final performance evaluation, thereby avoiding data leakage issues. This ensures that improvements in model performance are based on the inherent learning capabilities of model rather than due to improper data usage. Furthermore, more detailed information about the dataset can be found in Table 1.

2) EVALUATION METRICS

In the experiment, we use the mean reciprocal rank (*MRR*) and the hits at 1/3/10 (*Hits*@1/3/10) to evaluate the results of entity prediction and relation prediction on six public datasets. For the entity prediction task on WIKI and YAGO datasets, following the prior work related to RE-GCN [9], we only report the *MRR*, *Hits*@3, and *Hits*@10 results.

3) BASELINES

We primarily compare RPHF-GNN to static and temporal dynamic reasoning models. Static reasoning models include DistMult [20], ComplEx [21], ConvE [22], and ConvTransE [23]. Temporal dynamic reasoning models include TTransE [25], HyTE [8], RE-NET [10], RE-GCN [9], RE-GAT [15], Tlogic [16], RE-H^{2}AN [18], FITCARL [27], PPT [28] and DHE-TKG [17].

4) IMPLEMENTATION DETAILS

We exploited our RPHF-GNN model in PyTorch and trained it on a NVIDIA A800-PC. To ensure the convergence of the model, we determined the parameter configurations based on *MRR* performance obtained on the validation set, and set the training epoch to 500 for all datasets. After conducting extensive experiments, we set the embedding dimension to 200. The number of layers of R-GCN is two in ICEWS14, ICEWS18, ICEWS05-15, GDELT, and one in WIKI, YAGO.

TABLE 1. Details of the TKG datasets.

Dataset	$\|V\|$	$\|R\|$	$\|Train\|$	$\|Valid\|$	$\|Test\|$	Granularity
ICEWS14	6869	256	373018	45995	49545	24 hours
ICEWS18	23033	230	74845	8514	7371	24 hours
ICEWS05-15	10094	251	368868	46302	46159	24 hours
WIKI	12554	24	539286	67538	63110	1 year
YAGO	10623	10	161540	19523	20026	1 year
GDELT	7691	240	1734399	238765	305241	15 mins

The dropout rate for each layer of the R-GCN is set to 0.2. For ICEWS14, ICEWS18, ICEWS05-15, WIKI, YAGO and GDELT, we set the best historical length to 6, 6, 15, 2, 2 and 12 respectively. The parameter ψ is experimentally set to 10°, and α are experimentally set to 0.7, respectively. For ConvTransE, the number of kernels is set to 50, the kernel size is set to 2×3, and the dropout rate is set to 0.2. We use the Adam optimizer [37] for parameter learning with a learning rate of 0.001. Similar to RE-GCN, static graph constraints are added to ICEWS14, ICEWS05-15, and ICEWS18.

B. RESULT ANALYSIS

1) RESULTS ON ENTITY PREDICTION

Table 2 and Table 3 present the entity prediction performance of RPHF-GNN and baseline models on six fact-based datasets, with the best-performing models in bold and the second-best models underlined. In terms of models, RPHF-GNN maintained the best performance on most of the evaluation metrics across all six datasets. These results provide strong evidence of the effectiveness of the model. Static reasoning methods exhibit inferior performance compared to the RPHF-GNN model due to their inability to capture temporal fact information. In the interpolation setting, RPHF-GNN outperforms TTransE and HyTE because they do not consider historical data and lack the ability to capture relation dependencies. When compared to temporal dynamic reasoning models like RE-NET, RE-GCN and DHE-TKG in the extrapolation setting, RPHF-GNN still performs excellently. This is because these methods, which are based on historical information for reasoning and evolution, do not consider the interaction and evolution between historical and future sequence. They overlook the assistance that future information at a certain moment can provide for the model to adapt to new evolutionary patterns. Additionally, they fail to consider the dependency between entity embeddings and static properties, overlooking the adjustment of entity embeddings to static properties. This can affect the understanding of static properties by the model. In terms of datasets, RPHF-GNN has demonstrated exceptional performance when dealing with datasets of varying scales. In the WIKI and GDELT datasets, the abundance of training samples enables the model to accurately capture the evolutionary trends of entities and relationships. The large-scale data enhances the ability of model to identify key patterns, which is crucial for predicting future evolutionary trends. In datasets like YAGO, where entities and relationships are fewer, the model can also more delicately identify interactions and evolution trends between entities, as the reduction in data volume allows the model to focus more intently on learning the specific characteristics of each entity. This adaptability to smaller data sets is also reflected in the improved performance of the model. For the ICEWS series of datasets, the model, while perceiving evolutionary patterns, integrates historical information into static properties through the SP-Cell component, optimizing the static loss and further enhancing prediction results. Overall, RPHF-GNN demonstrates strong adaptability and predictive capabilities across various scales and characteristics of datasets compared to other baselines.

2) RESULTS ON RELATION PREDICTION

As shown in Table 4, RPHF-GNN performs better than most baselines. The results show that RPHF-GNN effectively improves performance on relation prediction tasks and achieves the best overall performance. RPHF-GNN demonstrates that by allowing the model to adapt to different evolutionary patterns, our module can obtain more accurate embedding representations. Given that the number of relations is much smaller than the number of entities, RPHF-GNN exhibits a relatively smaller performance improvement in relation prediction tasks. This is because in scenarios with a limited number of relations, the model has a finite set of features to learn, and thus its performance can readily reach a threshold. The performance improvement of RPHF-GNN in relation prediction on the YAGO and WIKI datasets is relatively modest. Upon further analysis of these datasets, we found that many relationships within them, such as 'was Born In', 'died In', and 'is Married To', tend to be more explicit, with weaker specific associations between facts. The method of enhancing relation prediction performance by sensing entities in the ''historical future'' may have limited effectiveness. In contrast, relationships in the ICEWS series and GDELT datasets, such as 'Make statement', 'Express intent to negotiate', and 'Engage in negotiation', involve various political and diplomatic interactions, which are likely more complex. The evolutionary trends of these relationships may be better captured by the model. Since some models are not designed for relation prediction, we only select ConvE

TABLE 2. Performance (in percentage) for the entity prediction task on ICEWS18, ICESW14 and ICEWS05-15.

Model	ICEWS14				ICEWS18				ICEWS05-15			
	MRR	H@1	H@3	H@10	MRR	H@1	H@3	H@10	MRR	H@1	H@3	H@10
DistMult	20.34	6.16	27.61	46.70	13.88	5.62	15.20	31.27	19.92	5.64	27.25	47.38
ComplEx	22.62	9.89	28.93	47.61	15.47	8.03	17.21	30.73	20.28	6.68	26.44	47.30
ConvE	30.31	21.30	34.42	47.88	22.81	13.63	25.83	41.43	31.40	21.56	35.70	50.97
ConvTransE	31.52	22.46	35.01	50.04	23.26	14.25	26.11	41.35	30.29	20.81	33.85	49.99
TTransE	12.86	3.14	15.73	33.66	8.44	1.86	8.98	22.39	16.56	5.55	20.78	39.28
HyTE	2.15	24.86	43.95	16.06	7.45	3.13	7.33	16.02	16.05	6.55	20.23	34.73
RE-NET	35.80	26.00	40.12	54.88	26.20	16.44	29.88	44.36	36.88	26.24	41.86	57.61
RE-GCN	41.59	30.92	46.67	62.67	30.84	20.17	35.23	51.90	46.60	35.12	53.13	68.14
RE-GAT	40.69	29.78	45.88	62.09	29.79	19.31	33.85	50.45	46.65	35.24	53.04	68.33
TLogic	41.81	31.72	47.24	60.55	28.52	18.77	32.71	47.99	45.99	34.51	52.88	67.42
RE-H^{2}AN	41.11	31.16	45.75	60.34	29.97	19.64	34.04	50.64	-	-	-	-
FITCARL	41.80	28.40	**52.20**	**68.10**	29.70	15.60	**38.60**	**58.40**	34.50	20.20	48.20	**73.20**
PPT	38.42	28.94	42.50	57.01	26.63	16.94	30.64	45.43	38.85	28.57	43.35	58.63
DHE-TKG	40.02	30.13	44.99	-	29.23	19.15	33.31	-	45.05	34.16	50.97	-
RPHF-GNN	**42.66**	**31.75**	47.99	63.79	**31.29**	**20.60**	35.61	52.23	**46.85**	**35.32**	**53.39**	68.59

TABLE 3. Performance (in percentage) for the entity prediction task on WIKI, YAGO and GDELT.

Model	WIKI			YAGO			GDELT			
	MRR	H@3	H@10	MRR	H@3	H@10	MRR	H@1	H@3	H@10
DistMult	27.95	32.43	39.53	44.06	49.70	59.96	8.62	3.93	8.22	17.02
ComplEx	27.63	31.92	38.60	44.08	49.51	59.66	9.84	5.15	9.55	18.21
ConvE	26.03	30.51	39.19	41.22	47.03	59.91	18.36	11.29	19.36	32.12
ConvTransE	30.88	34.30	41.44	46.62	52.23	62.53	19.05	11.88	20.33	33.16
TTransE	20.68	23.89	33.05	26.11	36.29	47.73	5.56	0.46	4.98	15.37
HyTE	25.41	29.17	37.55	14.45	39.75	46.99	6.71	0.01	7.58	19.06
RE-NET	30.85	33.56	41.23	46.82	52.72	61.93	19.59	12.03	20.53	33.89
RE-GCN	51.66	58.44	69.57	63.09	71.17	82.36	19.29	11.98	20.60	33.60
RE-GAT	-	-	-	-	-	-	19.11	11.80	20.44	33.34
RE-H^{2}AN	-	-	-	-	-	-	18.69	11.36	19.91	33.13
DHE-TKG	51.20	57.47	69.25	62.93	71.00	82.72	-	-	-	-
RPHF-GNN	**51.88**	**58.69**	**69.94**	**63.88**	**72.88**	**84.36**	**19.88**	**12.33**	**21.34**	**34.60**

TABLE 4. Performance (in percentage) for the relation prediction task.

Model	ICEWS14	ICEWS18	ICEWS05-15	YAGO	WIKI	GDELT
ConvE	38.81	37.73	37.89	91.33	78.25	18.86
ConvTransE	38.42	38.01	38.27	90.95	86.65	18.99
RE-GCN	40.80	40.97	**41.01**	93.76	97.94	19.21
DHE-TKG	40.11	39.42	39.55	**94.07**	**98.18**	-
RPHF-GNN	**41.02**	**41.05**	40.94	93.99	97.87	**19.68**

TABLE 5. Comparison of prediction time of extrapolation methods for all datasets (d: days; h: hours; min: minutes; s: seconds).

Model	ICEWS14	ICEWS18	ICEWS05-15	YAGO	WIKI	GDELT
RE-NET	3.07 min	23.15 min	19.88 min	8.23min	26.06min	32.6min
RE-GCN	**40s**	**3.07 min**	**27.18min**	**5s**	**25s**	**5.12min**
Tlogic	37.91 min	1.37 d	20.63 h	-	-	-
FITCARL	2.93 min	2.26 d	22.66 min	-	-	-
RPHF-GNN	1.83 min	5.75 min	52.53 min	14s	42s	27.5min

and ConvTransE from the static models, as well as RE-GCN and DHE-TKG from the temporal models, which can be used for relation prediction, and only the experimental results of MRR are displayed.

3) COMPARISON ON PREDICTION TIME

To explore the time efficiency of the model, we compared the prediction time of RPHF-GNN with that of publicly available extrapolation baseline methods across all datasets. As shown in Table 5, except for the ICEWS05-15 dataset, RPHF-GNN achieved the second-best performance in all datasets. RE-NET has lower time efficiency because it processes individual queries for each timestamp. Compared with the single-sequence recurrent modeling of RE-GCN, RPHF-GNN employs dual-sequence modeling, and it requires a longer historical sequence at each timestamp for training to reduce potential information loss, thus being more time-consuming. Tlogic is the most time-consuming because it requires learning a large number of rules, and for each rule length, multiple samplings are needed to generate logical rules. FITCARL guides the search process based on the traversal path through a reinforcement learning strategy

network, which increases the complexity of the model. In summary, RPHF-GNN provides excellent extrapolation performance while ensuring time efficiency.

4) SENSITIVITY ANALYSIS OF HISTORICAL LENGTH

The historical length refers to the span of learning about past knowledge by the model at each timestamp. To explore the importance of historical length on the TKG prediction results, we performed a sensitivity analysis for it. As shown in Figure 5, as the historical length gradually increases, the overall performance of the model also improves, indicating that RPHF-GNN can effectively learn from past information and translate it into higher prediction accuracy. However, when the historical length exceeds a certain threshold, it inevitably introduces a large amount of irrelevant entity information, leading to a decrease in accuracy. The results show that the model performs well in the ICEWS14 dataset when the history length is set between 5 and 9.

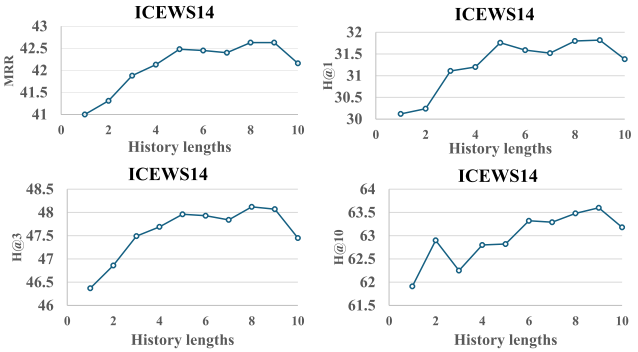

FIGURE 5. Historical length sensitivity analysis results in ICEWS14.

5) SENSITIVITY ANALYSIS OF TASK-WEIGHT

To explore the importance of task-weight (α) on prediction results, we performed a sensitivity analysis on the hyperparameter α, which serves as a variable balance factor between loss of entity and relationship. As shown in Figure 6, neglecting the weight of the losses of the entity or the relationship adversely affects the prediction results. Through extensive experimentation, we found that adjusting the task weight α to the range of 0.6 to 0.8 achieves an optimal balance between the entity and relationship predictions for RPHF-GNN. Given the significantly greater number of entities compared to relationships in the dataset, this phenomenon shows that the model requires a greater focus on entity prediction to maintain overall predictive performance.

6) ABLATION STUDIES

In this section, to explore the effectiveness of different components of the model in the evolution module, we performed ablation studies on each component based on the six data sets, as shown in Table 6. To assess the contribution of Hi-GRU to the final results, we replaced it with GRU and substituted the original TKG sequences in the model

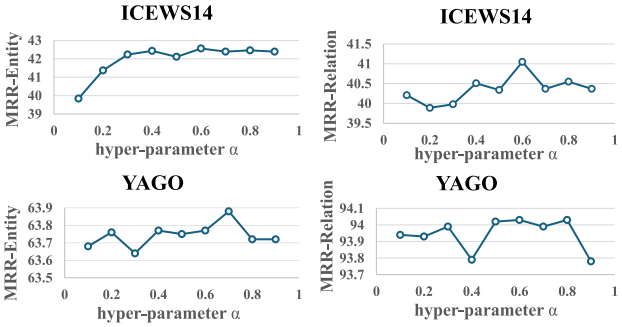

FIGURE 6. Task-weight sensitivity analysis results in ICEWS14 and YAGO.

TABLE 6. Results (in percentage) of ablation studies with MRR. hi is for Historical-future Information Gated Recurrent Unit, sp is for Static Perception Cell, ga is for Time-gate Integration Component.

Model	ICE18	ICE14	ICE05-15	WIKI	YAGO	GDELT
hi	31.07	41.34	46.63	**52.08**	63.79	19.79
sp	30.94	41.72	**47.06**	-	-	-
ga	31.10	42.18	47.01	51.90	63.72	19.82
sp+ga	31.18	42.10	46.91	51.65	63.72	19.86
hi+sp	30.70	41.47	46.88	51.99	63.79	19.81
hi+ga	31.01	42.24	46.95	51.97	63.73	19.84
RPHF-GNN	**31.29**	**42.66**	46.85	51.88	**63.88**	**19.88**

with subgraph combination sequences. It was observed that after replacing this component, the results decreased for all datasets, emphasizing the critical importance of capturing different evolution patterns in the future. Additionally, the combination of Hi-GRU and the improved temporal gate exhibited the best performance in addition to the model itself. This aligns with the notion that the weighting of the evolutionary component in the output of the graph convolution network from Section III can have a positive impact on Hi-GRU. Since we only established static graphs for the ICEWS14, ICEWS18, and ICEWS05-15 datasets, we applied SP-CELL exclusively to these three datasets. It was observed that using SP-CELL independently also improved the performance of the model. Thus, adjusting static properties by incorporating entity information during the evolution process helps alleviate the issue of information forgetting and aligns it with entity embeddings. This can have a highly positive impact on the predictive performance of the model.

V. CONCLUSION

This paper proposes a TKG reasoning model named RPHF-GNN. The model recursively models the sequential dependencies of entities and relationships in the 'historical past' while continuously extracting entity information related to the historical future' for parallel modeling, adapting to the changing trends of evolutionary patterns. Furthermore, the model captures the dependencies between entity embeddings and static properties during the evolutionary process, integrating historical information into static properties to optimize

static loss. Thus, the model employs a scoring function based on the evolutionary representations obtained from the final timestamp for TKG reasoning. Experimental results on six datasets show that RPHF-GNN has significant advantages in temporal entity prediction and relational prediction tasks. The results not only show a significant improvement in predictive accuracy, but also have profound implications for related fields, particularly recommendation systems, financial analysis, and healthcare. In recommendation systems, the improved predictive precision of our model allows a more accurate capture of user preferences and behavioral patterns, leading to more personalized recommendations and improved user experience and satisfaction. In the financial analysis domain, the RPHF-GNN model provides robust support for investment decisions and risk management by accurately predicting market trends and changes in entity relationships. In healthcare, by analyzing patient historical medical records and real-time health data, RPHF-GNN can predict disease progression and potential complications, assisting physicians in formulating more precise treatment plans. In summary, the RPHF-GNN model has not only achieved technical breakthroughs but also demonstrated broad application prospects and significant social value. Future work will further explore the application of the model in various fields and optimize and adjust it according to specific scenarios to achieve broader social benefits and technological advances.

REFERENCES

[1] E. Boschee, J. Lautenschlager, S. O'Brien, S. Shellman, J. Starz, and M. D. Ward, "ICEWS coded event data," *Harvard Dataverse*, Jan. 2015.

[2] X. Zhang, Y. Cheng, X. Bi, G. Yu, R. Cao, and L. Zhang, "TKMBR: Temporal knowledge graph-based multi-behavior recommendation for e-commerce," Tech. Rep., 2023.

[3] X. Zhu, L. Wang, H. Xin, X. Wang, Z. Jia, J. Wang, C. Ma, and Y. Zengt, "T-FinKB: A platform of temporal financial knowledge base construction," in *Proc. IEEE 39th Int. Conf. Data Eng. (ICDE)*, Apr. 2023, pp. 3671–3674.

[4] M. Luo, Y.-T. Wang, X.-K. Wang, W.-H. Hou, R.-L. Huang, Y. Liu, and J.-Q. Wang, "A multi-granularity convolutional neural network model with temporal information and attention mechanism for efficient diabetes medical cost prediction," *Comput. Biol. Med.*, vol. 151, Dec. 2022, Art. no. 106246.

[5] H. Dong, P. Wang, M. Xiao, Z. Ning, P. Wang, and Y. Zhou, "Temporal inductive path neural network for temporal knowledge graph reasoning," *Artif. Intell.*, vol. 329, Apr. 2024, Art. no. 104085.

[6] A. García-Durán, S. Dumančić, and M. Niepert, "Learning sequence encoders for temporal knowledge graph completion," 2018, *arXiv:1809.03202*.

[7] L. Bai, X. Ma, X. Meng, X. Ren, and Y. Ke, "RoAN: A relation-oriented attention network for temporal knowledge graph completion," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106308.

[8] S. S. Dasgupta, S. N. Ray, and P. Talukdar, "HyTE: Hyperplane-based temporally aware knowledge graph embedding," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2001–2011.

[9] Z. Li, X. Jin, W. Li, S. Guan, J. Guo, H. Shen, Y. Wang, and X. Cheng, "Temporal knowledge graph reasoning based on evolutional representation learning," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 408–417.

[10] W. Jin, M. Qu, X. Jin, and X. Ren, "Recurrent event network: Autoregressive structure inference over temporal knowledge graphs," 2019, *arXiv:1904.05530*.

[11] S. Feng, C. Zhou, Q. Liu, X. Ji, and M. Huang, "Temporal knowledge graph reasoning based on entity relationship similarity perception," *Electronics*, vol. 13, no. 12, p. 2417, Jun. 2024.

[12] A. Signorini, A. M. Segre, and P. M. Polgreen, "The use of Twitter to track levels of disease activity and public concern in the US during the influenza a H1N1 pandemic," *PLoS ONE*, vol. 6, no. 5, May 2011, Art. no. e19467.

[13] L. Zhao, "Event prediction in the big data era: A systematic survey," *ACM Comput. Surv.*, vol. 54, no. 5, pp. 1–37, Jun. 2022.

[14] Y. Xia, M. Zhang, Q. Liu, S. Wu, and X.-Y. Zhang, "MetaTKG: Learning evolutionary meta-knowledge for temporal knowledge graph reasoning," 2023, *arXiv:2302.00893*.

[15] Z. Li, S. Feng, J. Shi, Y. Zhou, Y. Liao, Y. Yang, Y. Li, N. Yu, and X. Shao, "Future event prediction based on temporal knowledge graph embedding," *Comput. Syst. Sci. Eng.*, vol. 44, no. 3, pp. 2411–2423, 2023.

[16] Y. Liu, Y. Ma, M. Hildebrandt, M. Joblin, and V. Tresp, "TLogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs," in *Proc. 36th AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 4, pp. 4120–4127.

[17] X. Liu, J. Zhang, C. Ma, W. Liang, B. Xu, and L. Zong, "Temporal knowledge graph reasoning with dynamic hypergraph embedding," in *Proc. Joint Int. Conf. Comput. Linguistics, Lang. Resour. Eval.*, 2024, pp. 15742–15751.

[18] J. Guo, M. Chen, Y. Zhang, J. Huang, and Z. Liu, "Hierarchical hypergraph recurrent attention network for temporal knowledge graph reasoning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2023, pp. 1–5.

[19] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2013.

[20] B. Yang, W.-T. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," 2014, *arXiv:1412.6575*.

[21] T. Trouillon, J. Welbl, S. Riedel, E. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2071–2080.

[22] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, "Convolutional 2D knowledge graph embeddings," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 1–8.

[23] C. Shang, Y. Tang, J. Huang, J. Bi, X. He, and B. Zhou, "End-to-end structure-aware convolutional networks for knowledge base completion," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, Jan. 2019, pp. 3060–3067.

[24] X. Mei, L. Yang, Z. Jiang, X. Cai, D. Gao, J. Han, and S. Pan, "An inductive reasoning model based on interpretable logical rules over temporal knowledge graph," *Neural Netw.*, vol. 174, Jun. 2024, Art. no. 106219.

[25] T. Jiang, T. Liu, T. Ge, L. Sha, B. Chang, S. Li, and Z. Sui, "Towards time-aware knowledge graph completion," in *Proc. 26th Int. Conf. Comput. Linguistics, Tech. Papers*, 2016, pp. 1715–1724.

[26] L. R. Medsker and L. Jain, "Recurrent neural networks," *Des. Appl.*, vol. 5, p. 2, Dec. 2001.

[27] Z. Ding, J. Wu, Z. Li, Y. Ma, and V. Tresp, "Improving few-shot inductive learning on temporal knowledge graphs using confidence-augmented reinforcement learning," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*. Springer, Jan. 2023, pp. 550–566.

[28] W. Xu, B. Liu, M. Peng, X. Jia, and M. Peng, "Pre-trained language model with prompts for temporal knowledge graph completion," in *Proc. Findings Assoc. Comput. Linguistics*, 2023, pp. 7790–7803.

[29] M. Schlichtkrull, T. Kipf, P. Bloem, R. V. D. Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. 15th Int. Conf.*, Crete, Greece. Springer, Jul. 2018, pp. 593–607.

[30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[31] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.

[32] D. Q. Nguyen, T. D. Nguyen, D. Q. Nguyen, and D. Phung, "A novel embedding model for knowledge base completion based on convolutional neural network," 2017, *arXiv:1712.02121*.

[33] S. Vashishth, S. Sanyal, V. Nitin, and P. Talukdar, "Composition-based multi-relational graph convolutional networks," 2019, *arXiv:1911.03082*.

[34] J. Leblay and M. W. Chekol, "Deriving validity time in knowledge graph," in *Proc. Companion Web Conf. Web Conf.*, 2018, pp. 1771–1776.

[35] F. Mahdisoltani, J. Biega, and F. M. Suchanek, "A knowledge base from multilingual wikipedias—YAGO3," Tech. Rep., 2024. [Online]. Available: http://suchanek.name/work/publications
[36] K. Leetaru and P. A. Schrodt, "GDELT: Global data on events, location, and tone, 1979–2012," in *Proc. ISA Annu. Conv.*, vol. 2, 2013, pp. 1–49.
[37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

SILING FENG received the Ph.D. degree from the University of Electronic Science and Technology of China, in 2014. She is currently a Professor and a Ph.D. Supervisor with the School of Information and Communication Engineering, Hainan University. Her research interests include intelligent computing, big data analysis, and intelligent recommendation.

ZIMIN YE was born in Hainan, China, in 2000. He received the B.S. degree from the School of Optoelectronic Information Science and Engineering, Anhui University of Science and Technology, China, in 2022. He is currently pursuing the M.S. degree with Hainan University. His research interests include temporal knowledge graph and natural language processing.

QIAN LIU received the B.S. degree from Beijing Jiaotong University, Beijing, China, in 2017, and the M.S. degree from Hebei GEO University, in 2021. She is currently pursuing the Ph.D. degree with the School of Information and Communication Engineering, Hainan University, Haikou, China. Her research interests include temporal knowledge graphs, natural language processing, and machine learning.

MENGXING HUANG (Member, IEEE) received the Ph.D. degree from Northwestern Polytechnical University, in 2007. He joined the Staff with the Research Institute of Information Technology, Tsinghua University, as a Postdoctoral Researcher. In 2009, he joined Hainan University. He is currently a Professor and a Ph.D. Supervisor of information and communication engineering. He is also the Vice-President of the State Key Laboratory of Marine Resource Utilization in the South China Sea and the Leader of Hainan Key Laboratory of Big Data and Smart Service. He is the author of five books, more than 150 articles, and more than 30 inventions. His research interests include big data and intelligent information processing, e-commerce and e-government, smart tourism and healthcare, cloud computing, and the Internet of Things (IoT). He is a Senior Member of the Chinese Computer Federation (CCF) and a member of the Information System Committee in CCF.